

PFSP con Algoritmos Evolutivos

Guía de defensa: conceptos del proyecto y mejoras implementadas sobre la solución base (Algoritmo Genético y Algoritmo Memético para el Permutation Flow Shop Scheduling Problem).

Parte 1 — Conceptos del proyecto

El problema (PFSP)

n trabajos deben pasar, en el mismo orden, por m máquinas. Cada trabajo tarda un tiempo distinto en cada máquina. Se busca la permutación (orden de los trabajos) que minimiza el makespan (C_{max}): el tiempo en que termina el último trabajo en la última máquina. El makespan se calcula con la tabla de tiempos de finalización $C(i,j) = \max(C(i-1,j), C(i,j-1)) + \text{tiempo de procesamiento del trabajo } i \text{ en la máquina } j$.

Semilla aleatoria y comparación justa

Los algoritmos usan números aleatorios en la población inicial, la selección, el cruce y la mutación. Fijar una semilla (random.seed) hace esa secuencia reproducible. En el experimento, AG y Memético corren con las mismas 30 semillas: para cada semilla, ambos parten de la misma población inicial, así que la diferencia en el resultado se debe solo al algoritmo (con o sin búsqueda local), no a la suerte del punto de partida. Esto habilita una comparación pareada (la que usa el test de Wilcoxon).

RPD (Relative Percentage Deviation)

$$RPD(\%) = (C_{max_obtenido} - C_{max_mejor_conocido}) / C_{max_mejor_conocido} \times 100$$

Mide qué tan lejos, en porcentaje, quedó una solución respecto al mejor valor conocido (UB) de esa instancia. $RPD=0\%$ significa que se igualó el mejor valor conocido. Se usa un porcentaje en vez del makespan crudo porque permite comparar instancias de tamaños muy distintos en una misma escala (ta001 tiene makespan ~1300, ta071 tiene makespan ~6000).

¿Por qué 30 semillas (repeticiones)?

Un algoritmo evolutivo es estocástico: una sola corrida puede tener suerte o mala suerte. Con 30 repeticiones se puede calcular un promedio y una desviación estándar confiables del RPD, y aplicar un test estadístico. 30 es una convención estándar en estadística y en la literatura de metaheurísticas que balancea confiabilidad estadística contra tiempo de cómputo — no es un valor derivado de un análisis de poder estadístico formal, que sería sobre-ingeniería para el alcance de este proyecto.

Las 3 instancias: chica, mediana y grande

Instancia	Tamaño	Rol
ta001	20 trabajos x 5 máquinas	chica
ta041	50 trabajos x 10 máquinas	mediana
ta071	100 trabajos x 10 máquinas	grande

Permite ver si la ventaja del Memético frente al AG cambia según el tamaño del problema. En los resultados originales, esa ventaja crece con el tamaño (mayor mejora relativa en ta071 que en ta001), porque en instancias grandes el AG puro tarda más en converger por sí solo.

Población y generaciones

Una generación es un ciclo completo: seleccionar padres, cruzarlos, mutar y formar la nueva población de $tam_poblacion$ individuos. Con $tam_poblacion=40$ y $num_generaciones=100$ se evalúan unas 4040 permutaciones distintas por corrida. Son valores típicos de la literatura de AG para problemas de este tamaño, no ajustados con un experimento de tuning propio (línea de trabajo futuro razonable).

Operadores del Algoritmo Genético

- **Fisher-Yates** (poblacion.py): genera permutaciones aleatorias para la población inicial, de forma que las $n!$ permutaciones posibles tengan la misma probabilidad de salir.
- **Selección por torneo binario** (seleccion.py): se eligen 2 individuos al azar y gana el de menor makespan. Presión de selección baja: favorece lo bueno sin eliminar toda la diversidad.
- **Cruce OX** (cruce.py): copia un segmento de un padre y completa el resto con el orden del otro padre, saltando lo ya copiado. Necesario porque un cruce de un punto normal produciría trabajos repetidos o faltantes en una permutación.
- **Mutación por intercambio (swap)** (mutacion.py): intercambia dos posiciones al azar. El resultado sigue siendo automáticamente una permutación válida.
- **Elitismo** (reemplazo.py): el mejor individuo de la generación actual reemplaza al peor de la nueva población si es mejor que este, garantizando que el mejor fitness nunca empeore entre generaciones.

Búsqueda local y frecuencia_bl (Algoritmo Memético)

El Memético agrega, cada frecuencia_bl generaciones, una búsqueda local por inserción (busqueda_local.py) al mejor individuo: prueba mover cada trabajo a cada otra posición y aplica la mejora (first-improvement: la primera que aparece). Es la parte de "explotación" que el AG puro no tiene. frecuencia_bl=50 en la versión original se eligió porque una sola pasada de búsqueda local cuesta ~43s en ta071 (100 trabajos): aplicarla más seguido haría que el experimento completo (180 corridas) tomara horas en vez de minutos.

Diseño experimental y test de Wilcoxon

experimento.py corre 3 instancias x 2 métodos x 30 semillas = 180 corridas, silenciando la salida de ag.py/memetico.py y guardando cada resultado en un CSV con flush inmediato (para no perder datos si el proceso se corta). analisis.py calcula el resumen estadístico y aplica el test de Wilcoxon (rangos con signo): compara, semilla por semilla, el RPD de AG contra el de Memético en la misma instancia. Se usa Wilcoxon y no un t-test porque no se asume que el RPD siga una distribución normal. Un p-valor menor a 0.05 indica una diferencia estadísticamente significativa entre ambos métodos.

Parte 2 — Mejoras implementadas

La solución base ya cumplía todos los requisitos del enunciado. Sobre esa base se incorporaron 4 mejoras concretas para acercar más las soluciones al mejor valor conocido (UB), manteniendo la interfaz por línea de comandos original.

1. Semilla NEH en la población inicial

Archivo nuevo: `src/neh.py`. Modificados: `ag.py`, `memetico.py`.

En vez de que los 40 individuos iniciales sean permutaciones 100% aleatorias, se agregó la heurística constructiva NEH (Nawaz, Enscore y Ham, 1983): ordena los trabajos de mayor a menor tiempo total de procesamiento y los inserta uno por uno en la posición que da el menor makespan parcial. Es determinista y da una solución de partida bastante mejor que una permutación al azar. Un solo individuo de la población inicial se reemplaza por la solución NEH; los otros 39 siguen siendo aleatorios para no perder diversidad. Verificado en `ta001`: NEH da $C_{max}=1286$ (RPD 0.63%) contra $C_{max}=1382$ (RPD 8.1%) del mejor individuo entre 40 generados al azar.

2. Mutación adaptativa (decreciente)

Modificados: `ag.py`, `memetico.py`.

`prob_mutacion` se interpreta ahora como valor inicial, y decae linealmente hasta el 10% de ese valor en la última generación. Más exploración cuando la población todavía es diversa, más explotación/convergencia hacia el final, en vez de una probabilidad constante durante toda la corrida.

3. Búsqueda local sobre los k mejores individuos

Modificados: `memetico.py`, `src/leer_parametros_memetico.py`.

Nuevo parámetro opcional `k_mejores` (octavo argumento de línea de comandos, por defecto 1: comportamiento original). Permite aplicar la búsqueda local a los k mejores individuos en cada aplicación, en vez de solo al mejor, explotando más de una zona prometedora del espacio de búsqueda a la vez. En el experimento final se usó `k_mejores=2`.

4. Frecuencia de búsqueda local adaptada al tamaño de instancia

Modificado: `experimento.py`.

La versión base usaba `frecuencia_bl=50` fijo para las 3 instancias. Se reemplazó por un valor por instancia (`ta001`: 10, `ta041`: 25, `ta071`: 50, sin cambios) para aprovechar que en instancias chicas una pasada de búsqueda local es casi instantánea, sin disparar el tiempo total del experimento en la instancia grande.

Parte 3 — Resultados finales

Comparación del RPD (%) obtenido antes y después de las 4 mejoras, sobre las 3 instancias y 30 semillas por combinación. "Mejor" es el RPD de la mejor de las 30 corridas (la solución más cercana al UB que se logró); "promedio" es el RPD medio de las 30 corridas, la métrica que reporta el test de Wilcoxon.

Instancia	Método	RPD mejor (antes)	RPD mejor (después)	RPD promedio (antes)	RPD promedio (después)
ta001	AG	1.33%	0.63%	2.49%	0.63%
ta001	Memético	0.00%	0.00%	1.41%	0.55%
ta041	AG	11.20%	4.35%	12.86%	4.78%
ta041	Memético	3.51%	2.47%	5.26%	3.55%
ta071	AG	7.49%	1.32%	9.43%	1.32%
ta071	Memético	0.99%	0.52%	1.69%	0.64%

El Memético queda por debajo del 3% de RPD en las 3 instancias, tanto en promedio como en la mejor corrida (0%, 2.47% y 0.52% respectivamente) — en ta001 llegó exactamente al óptimo conocido (1278). El único caso sobre 3% es el AG puro en ta041 (4.35%/4.78%), que es esperable: el AG es el método de exploración pura, sin explotación; el punto del proyecto es justamente mostrar que el Memético (exploración + explotación) lo supera, lo cual el test de Wilcoxon confirma con significancia estadística en las 3 instancias (p -valor < 0.05 : ta001 $p=0.041$, ta041 $p<0.001$, ta071 $p<0.001$).

Posibles preguntas de defensa

- ¿Por qué NEH y no otra heurística constructiva? Es la más citada en la literatura de PFSP, simple de implementar y de bajo costo (una construcción determinista, no repetida por generación).
- ¿Por qué no reemplazar toda la población con NEH? Se perdería diversidad: el AG necesita material genético variado para que el cruce combine características distintas.
- ¿Se probó best-improvement o un cruce alternativo (PMX)? Se prototipearon y se midió su costo: best-improvement resultó ~3x más lento por pasada en ta071 para una mejora marginal (~0.07% menos de RPD), así que no se justificaba frente al tiempo de cómputo disponible. Ambos se descartaron del código final para no dejar rutas sin usar.
- ¿Por qué 30 semillas y no más o menos? Es una convención que balancea confiabilidad estadística contra tiempo de cómputo, no un valor derivado de un análisis de poder estadístico formal.
- ¿Por qué el AG en ta041 queda sobre 3% de RPD? Es intencional: el AG es el método de referencia sin explotación, para mostrar el punto de comparación del proyecto. El Memético, que sí incluye explotación, es el que se mantiene bajo 3% en las 3 instancias.